

Predicting A User Rating From Goodreads Data Using Graphical Neural Networks

Anonymous
University of California, San
Diego
San Diego, California, USA

Anonymous
University of California, San
Diego
San Diego, California, USA

Anonymous
University of California, San
Diego
San Diego, California, USA

1 Dataset and Exploratory Analysis

With the rise of user-generated data on preferences, recommender systems have improved significantly. Here, we analyze data from Goodreads.com, a popular book recommendation platform. We describe the dataset and how its properties guide our model design.

1.1 Dataset

We use two datasets compiled by Wan and McAuley [5]. The first includes user IDs, book IDs, ratings (0–5 scale), optional review text, and votes on reviews. The second contains book-level data, including book IDs, number of pages, average ratings, number of ratings, text reviews, and genres. Genres follow Wan and McAuley’s classifications.

1.2 Exploratory Analysis

1.2.1 Basic Statistics. The dataset includes 31,220 books and 4,993,063 user-book ratings. The average book rating is 3.94 (range: 0–5), with *The Odyssey* being the highest-rated book with 680,823 ratings. The median rating is 3.96, with first and third quartiles at 3.74 and 4.18.

1.2.2 Directed explorations. We conducted a correlation analysis on the second dataset and found a strong correlation (0.789) between the number of ratings and review length, leading us to exclude the number of ratings due to collinearity. Moderate correlations were observed between genre and the other features (pages, average rating, text reviews, and genre), showing that genre emerged as a meaningful feature and suggesting that ratings might vary across genres. We predicted that ratings in different genres might have different distributions; some genres’ users might tend to have more polarized opinions than others. To capture this, we

added rating variance as a feature. Figure 1 shows correlations between all features, including rating variance. Boxplots in Figures 2 and 3 reveal genre differences: average ratings ranged from 3.89 (Mystery Thriller Crime) to 4.14 (Poetry, 0.25 difference), while variance ranged from 1.40 (Children’s) to 1.88 (Young Adult, 0.48 difference). The pronounced differences in variance led us to include it as a feature in our models.

2 Predictive Task

Our task involves a regressive prediction of the rating of each book in our dataset. Compared to the class material-focused datasets, we added more complexity to our data by not just including the user-book interactions and its rating, but also the publication year, number of pages, length of reviews, number of reviews, variance of the rating, and the genre. We also significantly increased the scale of the dataset by using 5 million total rows. All of these features will be used in our model excluding some baseline models. Since each of our models require different data processing steps, the data processing step is described in each model’s section. We implemented a 90/10 train/test split of our combined dataset. This allowed us to evaluate by calculating the root mean squared error (RMSE) between our models’ ratings prediction compared to the true labeled rating of each book. The value of the RMSE will allow us to track the validity of the model, with a higher RMSE indicating a less accurate model, and a lower RMSE indicating a more accurate model. We use several baseline models that are covered in the class, including item-based collaborative filtering with Jaccard similarity, regularized linear regression, and latent factor models. We will be comparing these models to our more complex model: the Graph Neural Network.

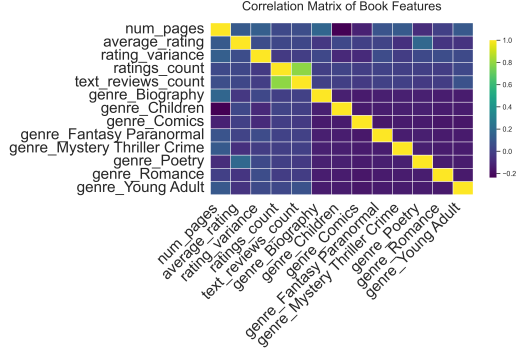


Figure 1: Correlation Matrix of Book Features

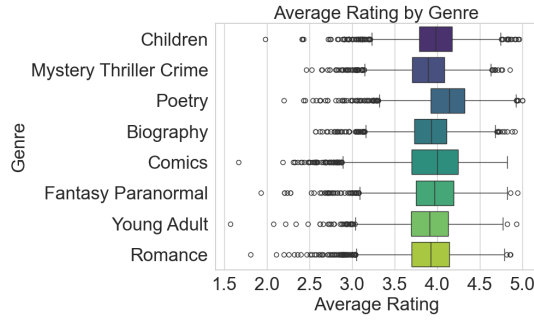


Figure 2: Average Rating by Genre

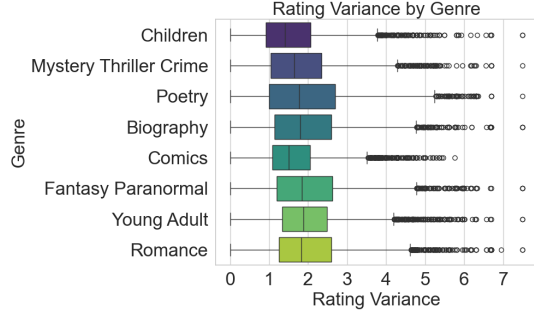


Figure 3: Rating Variance by Genre

3 Models

3.1 Baseline Models

3.1.1 Item-Based Collaborative Filtering. This straightforward approach incorporates the user ID, book ID, and rating using the user book interactions data table for ratings predictions. The model considers item-to-item similarity based on user rating patterns and computes the weighted average of normalized ratings of each book that the user had previously rated, where the weights represent how similar a book is to other books that the user had previously rated. The formula for the rating prediction is as follows:

$$\hat{r}_{u,i} = \bar{r}_i + \frac{\sum_{j \in N_i(u)} \text{sim}(i, j)(r_{u,j} - \bar{r}_j)}{\sum_{j \in N_i(u)} |\text{sim}(i, j)|}$$

Where the average rating of item i is calculated and are given higher weights for items that are more similar to item i . This is normalized by the total similarity weight, and adds onto the average rating.

The $\text{sim}(i, j)$ function represents the similarity method, of which we will implement two different similarity metrics:

First, we consider the Jaccard Similarity as follows:

$$J(i, j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|}$$

The Jaccard Similarity is used to determine how closely related all the books are in the data by counting the number of overlaps in user interactions between items divided by the total number of distinct books read between the two users.

Our second similarity function, the Pearson Correlation, is formulated as:

$$\rho(i, j) = \frac{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_i)(r_{u,j} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{i,j}} (r_{u,i} - \bar{r}_i)^2 \sum_{u \in U_{i,j}} (r_{u,j} - \bar{r}_j)^2}}$$

The Pearson Correlation measures a linear correlation between rating patterns. Different from the Jaccard similarity, the primary focus of the Pearson Correlation is to capture rating behavior patterns among users by comparing how they rate different items relative to their average ratings.

By incorporating book-based collaborative filtering as a baseline, we only select 3 columns in our dataset. While it is a weakness of the model that more features could provide more predictive power, we can use this as a baseline assess whether adding additional features in our other models will have a meaningful impact on improving performance.

However, one challenge we encountered in incorporating this model was the high computation time and high memory usage due to our user-book interactions dataset being comprised of 5 million observations. To resolve these issues, we implemented a batch processing technique and storing only the top 10 most similar items per book.

3.1.2 Regularized Linear Regression. While another simple approach, the regularized linear regression lets us incorporate all the features in our dataset while reducing overfitting on our testing dataset. The simple model's performance can be used to determine if a more complex model like the GNN would overfit on our training data. The regularized regression model is formulated as:

$$\min_{\beta} \|y - X\beta\|^2 + \alpha\|\beta\|^2$$

where the formula aims to minimize two objectives at once—the loss term:

$$\|y - X\beta\|^2$$

and the regularization term:

$$\alpha\|\beta\|^2$$

. Minimizing the loss term ensures that the model converges on the best "fit" for the training set. To avoid both under and over fitting, the regularized term is used to control the complexity of the model via L2 norm of coefficients. Setting the regularizing parameter α to 1 allows a balanced minimization of both objectives equally.

Ridge regularization helps balance model complexity and predictive accuracy to prevent overfitting. We set alpha to 1 to give equal weight to both complexity and accuracy. However, the algorithm may not fully capture the data's complexity without introducing collinear columns. We used numerical features like average rating, rating variance, and number of pages, with missing values imputed using the column mean. Dummy variables were created for the genre column, converting it into binary columns for each genre.

3.1.3 Latent Factor Models. For our last baseline model, we implemented a latent factor model, which decomposes our dataset into lower dimensional matrices and capture hidden patterns among user-item preferences.

The prediction formula is written as:

$$\hat{r}_{ui} = \mu + b_u + b_i + q_i^T p_u$$

$\hat{r}_{ui}$ represents the predicted rating for user u and item i and contains the dot product of the item latent factor vector and the user latent factor vector, which

represents how well a user's preferences match a book's characteristics.

This prediction formula is used in the optimization objective:

$$\min_{\theta} \sum_{(u,i) \in K} (r_{ui} - \hat{r}_{ui})^2 + \lambda(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2)$$

In short, the model aims to minimize the same terms as the regularized linear model. The loss term represents the squared difference between the actual and the predicted rating, and the regularization term represents the item and user biases and vectors. The higher the dot product between the user and item latent vectors, the more likely the user u is to like book i . The biases determine patterns for both the users and books ratings.

Here, we incorporate our user-book interactions as a feature, as well as numerical features (average rating, variance of rating, and the number of pages). We will use the binary feature classes created from our linear regression model, where each genre is classified as its own feature. No further transformation is needed.

Since most users only rate a small fraction of all the books in the dataset, the dataset is incredibly sparse. Because of this, a matrix decomposition technique like SVD can perform fairly well and would serve as a strong baseline for our primary model. In addition, this model can also regularize and thus is effective against overfitting. However, the model itself is not very interpretable and we would only extract our RMSE score from this model.

For the latent factor model parameters, we chose a very high number of factors to capture interaction patterns in our large training dataset of 4 million rows. We chose a default learning rate of 0.005, and a moderate all regularization of 0.02 and number of epochs to be 20. Since hyperparameter tuning was too computationally intensive, we chose these parameters to be the most reasonable as our model.

Overall, all of our baseline models are highly simplified and their parameters are default values. A limiting factor to why we did not implement optimized baseline models was due to the large scale of the dataset, making it difficult to run hyperparameter tuning techniques without our runtime crashing. Nevertheless, each of these models are effective baselines that can be used to compare against our primary model.

3.2 Graph Neural Networks

A way to think about books and user interactions is as a bipartite graph. Users and books would be the nodes, where the edges indicating user-item interactions. Interactions could be a user reading a book, rating a book, or just by adding them to their bookshelf. Then by using Graph Convolution Networks, we can further explore the interactions between the nodes.

We build our model with the edges representing the ratings a user gives a book. For a set of users U and items I , we want the model to output a score $score(u, i)$ for the user u on an item i .

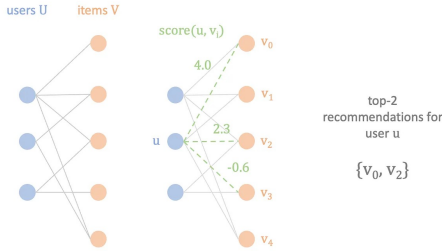


Figure 4: Graph diagram (from [GNN article](#))

The model focuses to learn embedding e_u for each user and for each item, an embedding e_i . However, we want to retain some parts of the features for users and books. By preserving some of the features for the user and books, we could react better for cold start situations. For a new item or user, we won't have their embeddings, if we could utilize the user and book features to provide some recommendations, then adaptive learning could be utilized for further refining the recommendations. We then try a few different configurations regarding the embedding space, layer depth, and the convolutions.

For the users and books, we have the following numeric features:

- average rating
- ratings variance

For the books, we have genre features, which contains 8 genres, and is a multi hot encoding label. Combining the two, we have the following overall score function:

1. **Score Function**:

$$score(u, v) = U^T \cdot B$$

2. **User Representation (U)**:

$$U = \frac{1}{L+1} \sum_{l=0}^L (e_{u,learned} + W_u \cdot u_{numeric})$$

3. **Item Representation (B)**:

$$B = \frac{1}{L+1} \sum_{l=0}^L (e_{v,learned} + W_v \cdot v_{numeric} + W_g \cdot v_{genre})$$

Where:

- $e_{u,learned}$: Learned embedding for the user.
- $e_{v,learned}$: Learned embedding for the item.
- $W_u \cdot u_{numeric}$: Projection of the user's numeric features.
- $W_v \cdot v_{numeric}$: Projection of the item's numeric features.
- $W_g \cdot v_{genre}$: Projection of the item's genre features.
- L : Total number of graph convolutional layers.

For the structure of our model, we utilize LightGCN as the convolution. There are a few benefits for this.

- **Stable performance**: When we compared the results to NGCF convolution, LightGCN training is much more stable
- **Efficiency**: Compared to other convolutions, we find that LightGCN's computational cost is relatively low, and is more efficient in memory usage.
- **Adaptation**: We could modify the structure easily for Top-K recommendation and other tasks based on the final loss function modification and the structure of the graph.

For the GNN model, we tried NGCF and LightGCN as the convolution layer. NGCF is more complex, including feature transformation matrices and interaction modeling. LightGCNConv is more simple and removes feature transformation. For the activation, NGCF uses a non-linear activation, increasing the complexity.

Comparing the results, we observed the following benefits using LightGCN:

- **More efficient**: LightGCN doesn't have non-linear activations, which lightens the compute cost
- **Performance**: Despite being more simple, LightGCN actually outperforms NGCF, also during the training process, we observed that LightGCN trains more stably

- **Less Prone to Overfitting:** We observed that NGCF is prone to overfitting, while LightGCN plateaus after a certain number of epochs.

For optimization of the models, we tried weighting mechanisms using weighted MSE during the initial testing as well as an attention mechanism. These two optimizations significantly increased the computational costs, while yielding minimum gains in performance. We then proceeded with the base LightGCN GNN recommendation system.

After obtaining a baseline, we optimized the following:

- **Batch and Memory optimization:** We modified the dataloader for more efficient sampling of neighbors
- **Gradient clipping and LR scheduler:** We implemented the gradient clipping and cosine annealing warm restarts during the training
- **Loss function:** In addition to the MSE loss, we added L2 reg and rating aware weighting, however, we found that the impact was minimal
- **Hyper parameter tuning:** We tuned the batch size, latent dimensions, number of layers and dropout rate

The final model architecture is shown in Figure 5.

4 Literature

The dataset originates from Goodreads.com and was preprocessed by Wan et al. [5]. Following the genre classifications in Section 1.1, we used categories such as Children’s books, Fantasy& Paranormal, Mystery, Poetry, Romance, and Young Adult. Books may belong to multiple genres based on user labeling.

Similar datasets from Goodreads, Amazon, and LibraryThing have been analyzed using machine learning methods like collaborative filtering and random forests. Random forests, known for their ensemble approach and robustness to noisy data, have outperformed baseline methods such as naive Bayes, K-nearest neighbor, and support vector machines [4]. These findings align with our results showing graph neural networks (GNNs) outperform simpler models.

GNNs, widely used in ranking prediction tasks [3], address issues like sparsity in collaborative filtering (CF) by leveraging knowledge graphs. Knowledge graphs enable information sharing between factors that traditional CF misses, improving predictive power [1]. Our

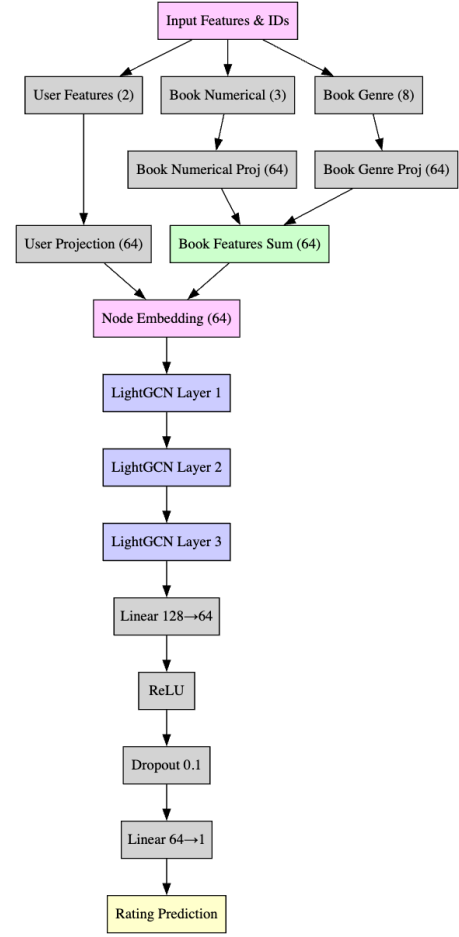


Figure 5: GNN model structure

findings similarly highlight GNNs as powerful tools for predictive modeling.

Large language models (LLMs), including GPT-based models, represent a newer state-of-the-art approach, achieving significantly lower RMSE in tasks like book rating prediction [6]. Combining GPTs with GNNs shows even stronger results [2]. While our models don’t reach GPT-level accuracy, their success reinforces the value of GNNs in predictive modeling.

5 Results

5.1 Performance of Models

Below we describe the results of the various baseline models as well as our final model.

5.1.1 Item-Based Collaborative Filtering. For our similarity collaborative filtering model, we had a significantly high RMSE of 4.121 for our Pearson Correlation,

and a much smaller RMSE of 1.383 for our Jaccard similarity. Given our rating scale, which is between 0 to 5, it seemed the Pearson wasn't a model that would be applicable in our scenario.

Table 1: Pearson Correlation Top 5 Similar Books to Harry Potter, Prisoner of Azkaban

Title	Similarity	Rating	Ratings
Metaphysical Dog	1.000	3.86	64
Expelled	0.852	2.14	7
The Etched City	0.850	3.86	14
The Pedestrians	0.844	3.43	28
Green Arrow #2	0.811	3.29	28

As shown in Table 1, we have very high similarity scores for books that seemed very similar to the fantasy-like book of Harry Potter and the Prisoner of Azkaban.

Table 2: Jaccard Similarity Top 5 Similar Books to Harry Potter, Prisoner of Azkaban

Title	Similarity	Rating	Ratings
Expectations of Happiness	0.220	2.70	20
A Fish to Feed	0.068	2.83	6
Shadow of What Was Lost	0.051	2.41	64
Time for Bed	0.036	4.00	790
Book Speak!	0.032	4.45	11

On the other hand, Table 2 shows us several children's books that are given the highest similarity scores to Harry Potter, which could be a true case, as Harry Potter could be classified as a children's fantasy. However, for either similarity score, we would usually expect to see more popular books to be similar to the Harry Potter series books.

5.1.2 Regularized Linear Regression. Our regularized linear regression model was the next best performer, with an RMSE score of 1.306 on the testing set. The importance scores are shown in Figure 6 - a significant amount of importance is given to the genre variables when determining the rating. We see that the importance of the features are very high for the genre labels as seen in Figure 6, indicating that the creation of a separate column as a binary variable for each genre was a meaningful feature representation to be used for our model.

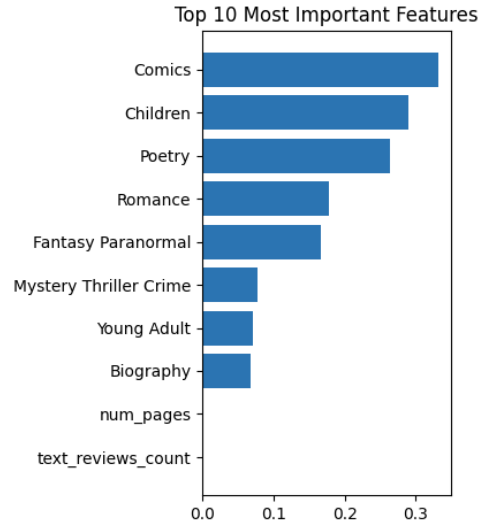


Figure 6: Linear Regression Importance Scores

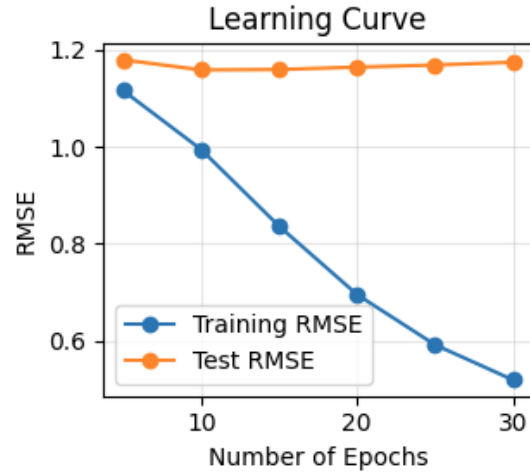


Figure 7: Latent Factor Model

5.1.3 Latent Factor Models. Our latent factor model was the best baseline performing model, with a test RMSE of 1.163. As shown in figure 7, the number of epochs do not affect the Test RMSE despite lowering the training RMSE.

5.1.4 Graph Neural Networks. The following is the graph plotting the training process of the GNN

For the graph neural network model, we achieved a final RMSE of 1.1185 and an MAE of 0.8154. For the GNN, we did hyper parameter tuning. To analyze the results, we could print out the model's representation of each node to analyze the similarity of the embedding

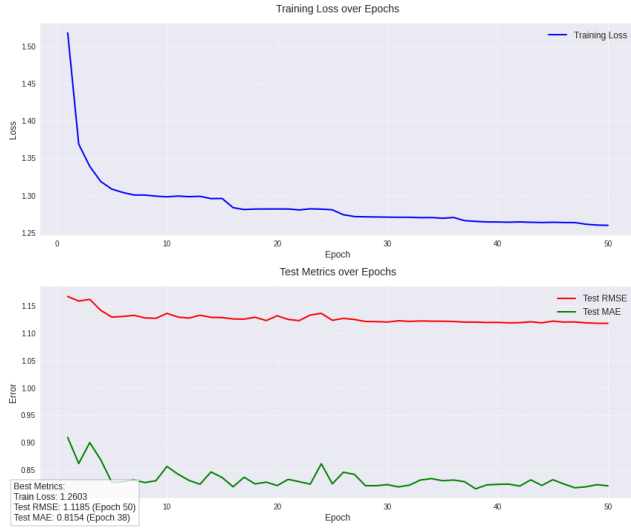


Figure 8: GNN performance on the final model

matrices. We could perform K-Means and group the nodes with similar embeddings into clusters. However, we could only analyze the results based on the genres we have since we don't have other features to categorize on. We discovered that the models were somewhat categorizing the similar nodes together, but with some other factors such as average ratings.

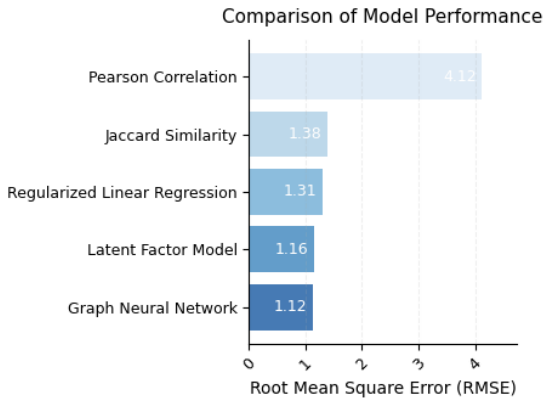


Figure 9: Comparison of RMSE Among Models

6 Discussion

Comparing the results, we see that the GNN model outperforms the other models. This is unsurprising as GNNs utilize larger number of parameters and propagate the embeddings to their neighbors along the edges.

GNNs can capture local structures as well as higher-order structures by using convolution. For K layers, each node is aware of the nodes that are k -hops away. However, as the number of layers increases, the computational cost increases. During the training, we started with $k=3$ and gradually increased the number of layers, hoping to find the optimal depth.

However, GNNs are quite time consuming to train. With our dataset of 30,000 books and 5,000,000 users, the GNN model took roughly 8 hours for 50 epochs (10 minutes/epoch). While the Latent Factor model only took 3-5 minutes to train. If we want to add new entries for the GNN model, we would need to do adaptive training, which could result in higher compute costs. Comparing the different GNN models, it is surprising that LightGCN, the more simple model, outperforms the more complex NGCF. This raises a very important question that might seem counterintuitive. We tend to believe that a more complex, more expressive model will yield better results. Why does LightGCN, a model with fewer learnable parameters outperform NGCF? We suppose this has something to do with the "actual" and "meaningful" weights. The weights in the user and books are significantly more important compared to the feature transformation. This results in LightGCN being able to focus on the more important parts of the model learning.

References

- [1] Shivangi Gheewala, Shuxiang Xu, Soonja Yeom, and Sumbal Maqsood. 2024. Exploiting deep transformer models in textual review based recommender systems. *Expert Systems with Applications* 235 (2024), 121120.
- [2] Shengxiang Hu, Guobing Zou, Song Yang, Yanglan Gan, Bofeng Zhang, and Yixin Chen. 2024. Large language model meets graph neural network in knowledge distillation. *arXiv preprint arXiv:2402.05894* (2024).
- [3] Xiao Li, Li Sun, Mengjie Ling, and Yan Peng. 2023. A survey of graph neural network based recommendation in social networks. *Neurocomputing* 549 (2023), 126441.
- [4] Itamar Reis, Dalya Baron, and Sahar Shahaf. 2018. Probabilistic random forest: A machine learning algorithm for noisy data sets. *The Astronomical Journal* 157, 1 (2018), 16.
- [5] Mengting Wan and Julian McAuley. 2018. Item recommendation on monotonic behavior chains. In *Proceedings of the 12th ACM conference on recommender systems*. 86–94.
- [6] Aakas Zhiyuli, Yanfang Chen, Xuan Zhang, and Xun Liang. 2023. Bookgpt: A general framework for book recommendation empowered by large language model. *arXiv preprint arXiv:2305.15673* (2023).